Manual y Guía de Defensa: Backend REST API

Caso de Negocio: Food DEI S.A. de R.L. | **Asignatura:** Programación Web (Ciclo 02, 2026)
Desarrollado por: Pablo Andrés Amaya Tenorio | **Tecnologías:** Node.js, Express, ES Modules

1. Resumen Ejecutivo y Arquitectura

El backend de **Food DEI S.A. de R.L.** es una API REST construida con **Node.js** y **Express** para gestionar reservas de almuerzos corporativos con porciones limitadas.

- **Almacenamiento:** Manejado en memoria mediante un archivo inicial `data/menu.json` cargado al inicio del servidor mediante Promesas asíncronas con `fs/promises`. La guía especifica: "*El servidor manejará en memoria (o en un JSON) un inventario de alimentos y el estado de la reserva del usuario*", por lo que la base de datos es opcional y no obligatoria.
- **Presupuesto del Usuario:** El cliente inicia con un saldo de `$25.00`. El backend descuenta el saldo y el stock disponible en tiempo real cada vez que se agregan o aumentan platillos.
- **Control de Desperdicio (Stock):** El restaurante cuenta con porciones estrictamente limitadas. Si se agotan, el backend rechaza la petición con código `400` y mensaje `"Stock insuficiente"`.
- **Límite de Tiempo:** Se implementó un control de reserva temporal con tiempo límite (300 segundos = 5 minutos) y endpoints de expiración y confirmación.

2. Flujo de Trabajo en Git (Gitflow)

Para cumplir con los criterios de evaluación de colaboración y orden de ramas:

1. `main`: Rama principal de producción del repositorio.
2. `develop`: Rama de desarrollo activa donde se integran las funcionalidades probadas.
3. `feature/Backend`: Rama donde se desarrolló, probó y aseguró la API de Node.js/Express, posteriormente integrada en `develop`.

3. Los 5 Endpoints Obligatorios de la Guía

Método	Ruta	Propósito y Regla Clave
GET	`/api/menu`	Devuelve la lista completa de platillos con id, nombre, precio, stock, categoría y disponibilidad.
GET	`/api/reserva`	Devuelve el estado de la reserva del usuario: `{ saldo, itemsReservados, total }` en tiempo real.
POST	`/api/reserva/agregar`	Añade temporalmente un platillo a la reserva. Valida que haya stock y que el cliente tenga saldo suficiente.
PUT	`/api/reserva/actualizar-porciones/:id`	Sube o baja porciones de un plato en el carrito. Si sube y no alcanza el saldo, devuelve 400 y mensaje específico.
DELETE	`/api/reserva/eliminar/:id`	Elimina el platillo de la reserva del usuario, devuelve las unidades al stock del restaurante y restaura el saldo.

Detalle de Peticiones y Respuestas

1. `GET /api/menu`


```
[
  { "id": 101, "nombre": "Menú Ejecutivo (Pollo)", "precio": 5.50, "stock": 12, "categoria": "Fuerte
  { "id": 102, "nombre": "Pasta Alfredo con Camarones", "precio": 6.25, "stock": 8, "categoria": "Fu
]
```


2. `GET /api/reserva`


```
{
  "saldo": 14.00,
  "itemsReservados": [
    { "id": 101, "alimentoId": 101, "nombre": "Menú Ejecutivo (Pollo)", "precio": 5.50, "cantidad":
  ],
  "total": 11.00,
  "saldoInicial": 25.00,
  "reservaActiva": true,
  "tiempoLimiteSegundos": 300
}
```


3. `POST /api/reserva/agregar`

Cuerpo enviado (JSON): `{ "alimentoId": 101, "cantidad": 1 }`

Validación de Stock (400 Bad Request):


```
{ "ok": false, "mensaje": "Stock insuficiente" }
```


Validación de Saldo (400 Bad Request):

```
{ "ok": false, "mensaje": "Saldo insuficiente para esta cantidad" }
```

4. `PUT /api/reserva/actualizar-porciones/:id`

Cuerpo enviado (JSON): `{ "cantidad": 3 }` (o también `{ "nuevaCantidad": 3 }`)

- Si `cantidad > actual`: Descuenta la diferencia del stock y valida que `costoExtra <= saldo`.
- Si `cantidad < actual`: Devuelve la diferencia al inventario y reintegra el saldo.
- Si `cantidad === 0`: Remueve el ítem de la reserva y devuelve todas sus unidades al stock.

5. `DELETE /api/reserva/eliminar/:id`

Remueve completamente el ítem del arreglo `itemsReservados`, recalcula el total y reincorpora el stock al menú.

4. Preguntas Frecuentes del Docente y Guía para la Defensa (40% de la nota)

Punto 1: ¿Por qué usaron 'let' y 'const'?

Usamos `const` para todas las referencias inmutables como la instancia de Express (`app`), el puerto (`PUERTO`), las funciones auxiliares y los objetos de configuración. Usamos `let` estrictamente para los estados que cambian durante la ejecución: el arreglo `inventarioMenu` (cuyo stock varía) y el objeto `estadoReserva` (donde cambian el saldo, los ítems y el total).

Punto 2: ¿Cómo se manejan los tipos de datos en JavaScript?

Se manejan tipados explícitos: **String** (nombre, categoría, mensajes de error), **Number** (precios con decimales, cantidades enteras, stock, saldo), **Boolean** (`disponible: stock > 0` , `reservaActiva` , `reservaConfirmada`) y **Object** (la estructura de cada plato y el estado global de la reserva).

Punto 3: ¿Qué métodos de arreglos (arrays) usaron y dónde?

Se aplicaron métodos funcionales de ES6 sobre los arreglos de datos:
• `.map()` : Para transformar y asegurar el formato del menú y los subtotales de ítems.
• `.find()` : Para buscar platillos por su ID en el menú y en la reserva.
• `.filter()` : En el endpoint DELETE y al llegar a 0 porciones para extraer un ítem del carrito sin mutar directamente.
• `.reduce()` : En `recalcularEstadoReserva()` para sumar el costo total de todos los platillos reservados: `itemsReservados.reduce((acc, item) => acc + item.precio * item.cantidad, 0)` .
• `.some()` : Para verificar si un alimento ya existía en la reserva antes de agregarlo.

Punto 4: ¿Dónde y cómo utilizaron Promesas?

Se demuestra en dos partes clave del código:
1) **Promesas Asíncronas nativas (`async/await`):** En `cargarInventarioInicial()` utilizando `readFile` del módulo `fs/promises` de Node.js para leer el archivo `menu.json` .
2) **Constructor `new Promise(resolve, reject)` :** En la función `verificarReglasNegocioAsync()` para encapsular la validación de inventario y saldo de manera asíncrona previo a confirmar cualquier modificación.

Punto 5: ¿Cómo garantizan que la API cumple con el estándar REST?

Se respetan los métodos HTTP según la semántica estándar (**GET** para consultar, **POST** para crear/agregar, **PUT** para modificar porciones por ID, **DELETE** para eliminar). Además, se retornan los códigos de estado correspondientes: `200 OK` para peticiones exitosas, `400 Bad Request` cuando se rompe una regla de negocio (stock insuficiente o falta de saldo), y `404 Not Found` cuando no existe el platillo solicitado.

5. Comandos para Levantar el Proyecto y Demostración

```
 # 1. Navegar a la carpeta backend
cd backend
```

```
# 2. Iniciar el servidor con recarga automática
npm run dev

# 3. En otra terminal, ejecutar las pruebas automatizadas:
npm run test:api
```